

Working on Eswachh

How the code is arranged, where every switch lives, what to edit for the changes you are most likely to be asked for, and the mistakes this codebase has already made so you do not repeat them.

STACK	Laravel 12 · PHP 8.2+ · Vue 3.5 · TypeScript 5.9 · Vite 7 · Pinia · TanStack Query · Tailwind 4 · MySQL
SUITE	429 tests, 1224 assertions, ~67s
COMPANION	admin-guide.pdf, for what the system does and why
REVISED	15 August 2026

What is in this handbook

-
- 01 Getting it running**
From clone to a working local copy

 - 02 How the code is arranged**
Filed by audience, not by resource

 - 03 Sector scoping**
Territory as tenancy, fail-closed, and why it answers 404

 - 04 Roles and abilities**
In code, not in tables — and custom roles on top

 - 05 Money**
Paise, PriceBook, and the two unbreakable rules

 - 06 Messaging**
The delivery gate, templates, and the dedupe key

 - 07 Every flag and switch**
What is off, where it is read, how to flip it

 - 08 Change this, edit that**
The tasks you will actually be handed

 - 09 The front end**
Two bundles, three routers, one trap in web.php

 - 10 Conventions**
Rules that hold everywhere in this codebase

 - 11 Tests**
Running them, and the two that guard the rest

 - 12 Commands and the schedule**
Everything artisan can do here

 - 13 Bugs this codebase has already had**
Read this one. All of these cost a day

 - 14 Cutting over from v1**
v1 is still taking orders. How to catch up, and what does not come
-

Getting it running

1.1

```
# everything: composer, .env, key, migrate, npm, build
composer setup

# the price list, message templates, policies, a first admin
php artisan db:seed

# server + queue + log tail + vite, all four at once
composer dev
```

`composer dev` runs `php artisan serve`, the queue listener, `pail` and `npm run dev` together and kills all four when any one stops. Use it rather than four terminals.

What must be in .env

KEY	WHY
<code>APP_TIMEZONE=Asia/Kolkata</code>	Not optional. v1 ran on it. On UTC, imported timestamps are mislabelled by five and a half hours and date filters silently find nothing.
<code>DB_CONNECTION=mysql</code>	The importer reads a v1 MySQL database directly.
<code>SESSION_DRIVER=database</code>	Sanctum SPA cookie auth.
<code>QUEUE_CONNECTION=database</code>	Messaging is queued.

ONE COMMAND TO ANSWER "IS IT WIRED UP?"

`php artisan eswachh:check-integrations` reports whether payments and messaging are configured and, when they are not, exactly what is stopping them. Add `--ping` to prove the gateway key actually works.

Importing v1 data

`app/Console/Commands/ImportLegacyData.php`

`php artisan eswachh:import` brings across customers, vehicles, subscriptions and payments, links every customer to a login that still accepts their old password, and issues invoice numbers for payments that had none. Five v1 orders cannot come across: their customers were deleted in v1, so there is nobody to attach them to.

Onto an empty database it is three commands, and the order is the whole of it:

```
php artisan migrate      # the tables
php artisan db:seed      # an administrator, the site's words, the message wording
php artisan eswachh:import # everything else, out of v1
```

Seeding before importing is right: the seeder creates no masters and no customers, only what a database needs before anybody can sign in.

THREE WAYS THIS GOES WRONG, ALL NOW CHECKED UP FRONT

The database was never migrated. The first thing the command does with any record is ask the legacy map whether it has been seen before, so the error named `legacy_references` — as though that one table had failed to migrate, when in fact none of them existed. It now says so plainly and prints the three commands.

LEGACY_DB_DATABASE pointed at a v2 database. An old working copy, a backup, or the database the command last imported `*into*` — all of them connect perfectly well, and several have a `states` table of their own, so the import gets some way in before anything looks wrong. It now checks for `orders`, `carcategories` and `payment_history` first: v1 kept subscriptions in `orders`, and v2 has no such table.

v1 is not one schema. `payment_id`, `razorpay_order_id`, `verified_by` and `verified_at` were added to `payment_history` part way through its life. Reading one directly on an older backup was a fatal error seven steps in, with the customers, cars and plans already written — the worst place to stop. Optional columns go through `column()` and a missing one means blank, which is what it meant in v1.

`--dry-run` reports without writing, `--step=` runs one stage, and `--fresh` wipes what was imported and starts again. Note that a dry run reports every payment as "no matching subscription": nothing is written, so nothing is in the map for the payments step to find. That is the dry run being honest, not a problem with the data.

How the code is arranged

- 2.1 Everything is filed by **who it is for**, not by what kind of file it is. The same four names appear on both sides of the wire.

AUDIENCE	BACKEND	FRONT END
The office	<code>app/Http/Controllers/Api/V1/Admin</code>	<code>resources/js/admin</code>
A customer	<code>.../Api/V1/Portal</code>	<code>resources/js/portal</code>
A visitor	<code>.../Api/V1/Site</code>	<code>resources/js/site</code>
More than one	<code>.../Api/V1/Shared</code>	<code>resources/js/shared</code>

Why not group by resource

A `SubscriptionController` holding everything anyone can do to a subscription hides the only question that matters here: *who is asking*. A customer holds `view.subscription` so they can look at their own plan. An ability cannot say "their own"; only a query can. Filing by audience puts that question in the folder name.

SHARED IS THE LOAD-BEARING ONE

Those controllers are reached by both the office and a customer, so each carries a row-level check inside it (`RestrictsToOwnRecords`). If one is ever moved into `Admin`, that check starts to look redundant to whoever edits it next — which is exactly how a customer ends up reading the whole branch's payments.

2.2

What is not in that table

FOLDER	HOLDS
<code>app/Domain/*</code>	The business itself: pricing, billing, messaging, the daily round, cloth ledger, complaints, alerts, phone codes. No HTTP, no request objects. A controller validates and delegates; the rules read the same whether a person, a scheduled job or a test called them.
<code>app/Support/*</code>	Plumbing that is not a business rule: <code>Tenancy</code> , <code>Access</code> , <code>Http</code> (sorting whitelists, ownership), <code>Settings</code> , <code>Numbering</code> , <code>Content</code> (sanitising), <code>Masters</code> , <code>Legacy</code> , <code>Database</code> , <code>Preferences</code> .
<code>app/Models</code>	One flat folder on purpose. Models are referenced from everywhere, and a <code>Subscription</code> filed under <code>Admin</code> would be a lie.

Adding a route

1. Decide who it is for. That is the folder.
2. If the answer is "both the office and a customer", it is `Shared`, and it needs an ownership check — `restrictToOwnRecords()` for a list, `ownsRecord()` for a single row, returning **404** and not **403**, because refusing must not confirm that the record exists.

3. Abilities go on the route or at the top of the method. They answer *may this person use this feature*, never *whose rows are these*.

```
routes/api.php – everything behind auth
routes/api_public.php – quotes, signup, renewal lookup, catalogue, blog
routes/web.php – which blade shell serves which URL
routes/console.php – the schedule
```

Sector scoping

3.1

app/Support/Tenancy/SectorContext.php · app/Models/Concerns/ScopedToSectors.php

A **sector is the territory**, and it is the whole of tenancy. Staff are assigned sectors through `user_sector`; a customer sits in one sector; a record is visible when those two meet.

```
users -< user_sector -> sectors -< customers -< subscriptions -< service_logs
      ∴ vehicles, complaints, cloth_entries
```

One rule: **a customer is visible to whoever covers the sector the customer sits in**. Nothing is copied onto the customer, so reassigning a sector is one pivot row and takes effect on the next request.

```
SectorContext::currentSectorIds()    // null for an admin, their ids otherwise
SectorContext::currentCustomerId()  // set only when the viewer IS a customer
SectorContext::isRestricted()        // false only for super_admin
SectorContext::withoutScope(fn () => ...) // masters, cross-sector checks
SectorContext::forSectors($ids, fn () => ...) // jobs, importers, tests
SectorContext::forget($userId)       // after their assignment changes
SectorContext::reset()                // tests, in setUp and tearDown
```

CURRENTSECTORIDS ANSWERS NULL INSIDE WITHOUTSCOPE

Null means "ask nothing of this query", which is right for the scope and wrong everywhere else. Called inside an unscoped block it turned every franchise user's own territory into an empty list on `/me`. Resolve it *before* lifting the scope.

What each model declares

Put `use ScopedToSectors;` on a model and it reduces to "which customers may this viewer see". Three tables do not and say so in `sectorScope()`:

TABLE	SCOPED BY
payments	A stamped <code>sector_id</code> , written at capture and never recomputed. A payment records something that happened, so who took the money does not change when territory is rearranged — hand a sector over and the new holder gets the customers and the plans, but last year's figures do not change shape.
attendances	<code>cleaner_id</code> against the staff covering the viewer's sectors — the same pivot, asked from the other end.
sectors	Their own id: you see the ones you are assigned.

404, NOT 403

A record in another sector must produce the same response as one that never existed. A 403 confirms there is something there to be refused, which is itself a leak — it tells a franchise user that a car number belongs to somebody, somewhere.

3.2

Three rules that are easy to get wrong

- **Fail closed.** Covering nothing returns nothing, never everything. That is the difference between a quiet screen and a leak.
- **A customer with no `sector_id` belongs to nobody** until one is given. SQL `IN` never matches null, so the rule needs no special case — but `sector_id` is therefore *required* when the office creates a customer, or it silently makes an invisible one.
- **A customer is never in the pivot.** Their territory comes from their address; an assignment would let them see their neighbours. The scope asks `currentCustomerId()` first and narrows to their own records instead.

Assigning territory

On the **person**, under People — that is the moment it matters, because an account created without a sector signs in to empty screens. The Sectors screen reports who covers each one but does not edit it.

Nobody hands out territory they do not hold: a franchise owner adding a cleaner can only give them sectors from their own, or assigning a colleague elsewhere would be a way to read another franchise's customers through them.

When to use `withoutScope`

Only for things that are genuinely shared, and always with a comment saying why: **masters** (every franchise sells from the same price list, and this is why a quote works on the public site with nobody signed in), and **uniqueness of a car number**, since a car on another sector's books is still taken.

The scope's own subqueries use the query builder rather than Eloquent, deliberately: they run inside the global scope that would otherwise call them again, and they must see soft-deleted rows so withdrawing a society does not hide its customers from the person still servicing them.

What this replaced

A `branch_id` copied onto the customer and onto everything of theirs. That column was a cache of where somebody lived, and it went stale: a sector was pointed at another franchise and two hundred customers stayed in the old owner's books while the sector claimed otherwise, with nothing anywhere reporting the disagreement. The columns are still on the tables, nullable and unread, until the model has run long enough to be trusted.

Roles and abilities

4.1

`app/Enums/UserRole.php` · `app/Support/Access/Abilities.php` · `app/Models/CustomRole.php`

Four roles, declared in code rather than in tables: `super_admin`, `franchise_owner`, `cleaner`, `customer`. Abilities are named `{verb}.{resource}` and listed on the enum. `super_admin` is granted everything by a `Gate::before` hook, which keeps the table readable.

WHY IN CODE

v1 stored roles and permissions in tables and then cached the lookup — a cache that shadowed the relation and left newly created users with no roles at all until somebody cleared it by hand. Abilities declared in code cannot drift out of sync with themselves, cannot be cached wrongly, and are visible in review.

Adding an ability

1. Add the string to the relevant arm of `UserRole::abilities()`.
2. Add it to `Abilities` so the roles screen can offer it.
3. Check it on the route or at the top of the method.
4. If it is a screen, add it to the `navigation` array in `resources/js/admin/AppLayout.vue` and to the route's `meta.ability`.

NEVER NAME AN ABILITY "OWN"

An ability answers *may this person use this feature*. Which rows they see is a separate question that only a query can answer. Naming one `view.own.complaint` implied otherwise and left routes checking `view.complaint` locking customers out of their own complaints entirely.

Custom roles

DB-backed, and each **refines** one built-in role: it starts from that role's abilities and may only narrow or pick from within them. A custom role cannot grant cross-branch sight; there is no way to express it. Managing roles is gated on *being* `super_admin`, not on an ability, so that a role cannot be used to grant the power to rewrite roles.

Money

5.1

app/Domain/Pricing/PriceBook.php · Quote.php · QuoteLine.php

```
( category + package + service_type + society surcharge ) × months
  - duration discount
  + cloth bundle
```

Every master is looked up fresh on each call. A quote is a statement about the price list *as it stands right now*, and must never be assembled from ids the caller has already resolved to amounts. A master that was asked for and cannot be sold produces a warning rather than being skipped — skipping it silently under-prices the plan by whatever it used to charge, and nobody would notice. Withdrawn and never-existed are told apart, because they need different fixes.

RULE ONE · NO AMOUNT EVER COMES FROM A REQUEST

Every price is worked out by `PriceBook`, on the server, at the moment it is charged. The figure the browser shows is a quote and is thrown away. v1 read `final_price` straight from the form and validated only `min:1` — a customer could have paid ₹1 for any plan.

There is a test for this on every paying route. If you add one, add the test: post a wrong `amount_paise` and assert the stored amount ignored it.

RULE TWO · NOTHING IS MARKED PAID EXCEPT A VERIFIED CALLBACK

Signup, renewal, top-up and add-a-car all open a payment and stop.

`Domain\Billing\RecordPayment` moves it, and only after the Razorpay signature checks out *and* the gateway confirms what it holds.

5.2

The billing pieces

CLASS	DOES
<code>StartPayment</code>	Opens a payment row and a gateway order, before the window appears, so an abandoned checkout still leaves a record.
<code>RecordPayment</code>	The only thing that marks anything paid. Idempotent.
<code>RazorpaySignature</code>	HMAC verification, constant-time.
<code>RazorpayGateway</code>	The HTTP client. Reports itself unavailable unless enabled and keyed.
<code>InvoiceNumber</code>	Issues a numbered invoice per captured payment, prefix from settings.

Paise, everywhere

All money is an **integer number of paise**, named `*_paise`. Never a float, never a decimal column, no exceptions. Format for display at the edge only.

Repricing on renewal

```
app/Http/Controllers/Api/V1/Shared/RenewalController.php
```

A customer renewing from their account may change package, service type and duration. The controller reprices and writes the new amount *before* opening the payment. Without that step a customer switching one month to six would be charged for one.

Messaging

6.1

app/Domain/Messaging/Messenger.php · app/Enums/MessagePurpose.php · database/seeder/Mess
geTemplateSeeder.php

The delivery gate

```
public function deliveryEnabled(): bool
{
    // Never during tests, whatever the configuration says.
    if (app()->runningUnitTests()) {
        return false;
    }

    return app()->isProduction() && (bool) config('services.whatsapp.enabled');
}
```

THIS IS THE LINE THAT MATTERS

A message leaves the building only when the app is in production *and* `WHATSAPP_ENABLED=true`. Everywhere else it is written to the `messages` table exactly as it would have been sent, together with a `suppressionReason()` saying why it was held. Demonstrations against real data cannot message a real customer.

Do not add a bypass for "just this one". If you need to see delivery work, point a staging copy at a test number.

Wording lives in the database

Templates are seeded with the exact wording from the requirements document and edited in the office. A template that is missing or switched off sends **nothing**, rather than falling back to wording nobody has approved. Both the nightly jobs and hand-sent messages read the same row.

The dedupe key

`MessagePurpose` values are load-bearing: the purpose is half of the key that stops a customer being messaged twice about the same thing. **Renaming a case silently allows a duplicate.**

`isPerEvent()` splits the two behaviours. A renewal reminder is one per customer per day. A "cleaning done" is per event, because a household with two cars must get one for each.

Every flag and switch

- 7.1 Three kinds. Environment variables are for credentials and things that differ per server. Settings are for the business. Hard-coded conditions are for what the business asked to hide.

Environment · .env

KEY	DEFAULT	READ BY
RAZORPAY_ENABLED	FALSE	RazorpayGateway::available()
RAZORPAY_KEY / _SECRET	empty	config/services.php
WHATSAPP_ENABLED	FALSE	Messenger::deliveryEnabled()
MSG91_AUTH_KEY	empty	Messenger
MSG91_WHATSAPP_NUMBER	empty	Messenger
APP_TIMEZONE	Asia/Kolkata	Everything. See section 13.

Settings · Settings screen, or SiteSettings::put()

app/Support/Settings/SiteSettings.php – the whitelist of keys

KEY	DEFAULT	EFFECT
cloth_service_enabled	0	Hides the top-up page, the cloth screens and the cloth choice on the forms. Touches no balance or ledger entry.
lock_plan_edits_to_admin	0	Franchise owners cannot change a plan. Never restricts an administrator.
seo_index	1	Off writes <code>noindex</code> . Switch it off on staging.
cloth_low_threshold	5	The warning list and the low-balance message.
renewal_grace_days	7	Days overdue before a plan is put on hold.
admin_notify_phone	empty	Where "a new car needs a cleaner" goes. v1 had this number written into two controllers.
invoice_prefix	ESW	Invoice numbering.
seo_title / _description / _keywords / _share_image	set	Rendered server-side into <code>site.blade.php</code> .
privacy_policy / terms / refund_policy	written	Rich text, sanitised on write.
business_name, legal_name, gstin, address, contact_*, whatsapp_number, office_hours, invoice_footer	various	Presentation.

Hidden in code

WHAT	WHERE	TO RESTORE
Renew & Subscribe hidden from a signed-in customer	<code>resources/js/site/PublicLayout.vue</code> → <code>showVisitorActions</code>	Change the computed to <code>true</code> . Both the wide header and the narrow drawer read it.
Cloth top-up not in the public header	<code>resources/js/site/PublicLayout.vue</code> → the <code>links</code> array	Add <code>{ name: 'Cloths', to: { name: 'cloth-top-up' } }</code> . The route and page are untouched.
Car number editable by administrators only	<code>.../Admin/SubscriptionEditController.php</code> → <code>actorMayRenumber()</code>	Deliberate, from the requirements document. A refused change sets <code>\$registrationRefused</code> so the response says so rather than staying silent.
Paid amount never editable	<code>.../Admin/SubscriptionEditController.php</code> → <code>applyPaymentChanges()</code>	Method, reference, paid-at and notes are editable; the amount is not. Do not add it.

Adding a setting

1. Add the key to `SiteSettings::definitions()` with a label, group, rules and default. Mark it `'boolean' => true`, `'rich' => true` or `'long' => true` as needed — the settings screen renders the right control from that.
2. Read it with `SiteSettings::get('your_key')`.
3. Nothing else. The cache rebuilds itself when the definition count changes, so a new setting does not need a cache clear to appear.

Nothing secret goes in settings. Gateway keys and API tokens stay in `.env`, where they are not inside a database dump an administrator can download from the Backups screen.

Change this, edit that

8.1

YOU ARE ASKED TO...

GO HERE

Change a price

Masters screen. Nothing in code. Prices are `*_paise` integers in the masters tables.

Change how a price is assembled

`app/Domain/Pricing/PriceBook.php` — and add a test, because historic amounts must still reproduce.

Change message wording

Messages screen. To change the *seed*:
`database/seeder/MessageTemplateSeeder.php`.

Add a new message

A case on `MessagePurpose` (+ `label()`, and `isPerEvent()` if it is per event), a row in the seeder, and the call site.

Add an office screen

A view in `resources/js/admin/views`, a route with `meta.ability` in `admin/router.ts`, an entry in the `navigation` array in `AppLayout.vue`, and a controller under `Api/V1/Admin`.

Add a customer portal screen

Same, but `resources/js/portal` and the `/my` children in `admin/router.ts`. It ships inside the admin bundle — see section 09.

Add a public page

`resources/js/site/views` + `site/router.ts`. Check the exclusion list in `routes/web.php` if the path collides.

Change the home banner

Banners are records, not code: `Banner` model, managed in the office. Layout is `resources/js/site/BannerSlider.vue`. The shipped images are generated — see 08.3.

Change the renewal wording

`resources/js/shared/RenewalNotice.vue`, which every renewal screen renders. The facts come from `Subscription::renewalTiming()`; only the sentences live here.

Change when a renewed term starts

`Subscription::continuesFromEndDate()`. One method, read by both the payment and the notice — see 08.4.

Add a screen that takes money

Call one of the `payFor*` helpers, or `openThenPay` if it is a new endpoint. Do not call `api.post` then `completeCheckout` by hand — see 08.5.

Switch a whole feature off

`App\Support\Settings\Features` — a setting, a middleware alias, and one flag reaching both front ends. See 08.6.

Save something on a screen

Finish with `refreshAfter(queryClient, 'people')`, never a bare `invalidateQueries` — see 08.8.

Change what a receipt says

Messages screen for the wording: `resources/views/receipt.blade.php` for the document; `Domain\Billing\Invoice` for what is on it.

Change SEO

Settings screen. Rendered server-side by `resources/views/site.blade.php` — it must be server-side or a crawler will not see it. See 08.2.

YOU ARE ASKED TO...	GO HERE
Link a dashboard tile	The <code>to:</code> on the tile in <code>resources/js/admin/views/DashboardView.vue</code> . Only link a tile whose count a filter can actually reproduce.
Change the policy pages	Settings screen, rich text. Shipped wording lives in <code>app/Support/Settings/PolicyText.php</code> .
Change retention	<code>app/Console/Commands/PruneServiceHistory.php</code> — currently fifty days, as the requirements document asks.
Change a scheduled time	<code>routes/console.php</code> . Read the comments first; the order is deliberate.
Add a sortable column	<code>app/Support/Http/SortsLists.php</code> — sorting is whitelisted, so an unlisted column is ignored rather than injected.
Add a master type	<code>app/Support/Masters/MasterRegistry.php</code> — the Masters screen builds itself from it.

8.2

What the server puts in the head

The site is a SPA, so everything a crawler or a link preview reads has to be in the HTML that arrives — nothing added by Vue afterwards counts. All of it is built in `resources/views/site.blade.php`, from the URL path and the database, before a byte of JavaScript runs.

- **A title per page**, from a `match ($section)` on the first path segment. Add a public route, add its arm — otherwise it inherits the site-wide title and competes with the home page.
- **A `LocalBusiness` block** from `SiteSettings` and the price list.
- **An `FAQPage` block** from the live `faqs` rows, with `strip_tags` applied — a rich result renders text, and markup in it is a malformed result.

TWO TRAPS, BOTH ALREADY SPRUNG

Do not build the JSON with `@json` on a multi-line array. Blade's parser gives up on the newline inside the brackets: *"Unclosed '[' on line 113"*. Build the array in an `@php` block and echo it with `{!! $asJson($data) !!}`.

Strip the nulls. An empty `telephone` in structured data is a claim there is no phone number, which is worse than the key being absent. `array_filter` on the way out.

Nothing here is visible in a browser, so a change that quietly drops a block goes unnoticed until somebody views the source. `tests/Feature/Site/SeoTest.php` is the only thing watching it — including that `seo_index` off withdraws the questions as well as the pages.

8.3

The banner images are generated, not bought

The four in `public/images/banners/beaded-*.jpg` are rendered from `docs/banners/hero.html` — water beading on a lit panel, four palettes off one composition:


```
chrome --headless=new --window-size=2400,1000 \  
    --screenshot=beaded-dusk.png "docs/banners/hero.html?p=dusk"  
# then dawn, monsoon, night – and convert to JPEG at ~q84
```

The bead positions are seeded off the palette name, so re-rendering reproduces the same image rather than quietly issuing a different one. Each is drawn dark down the left, because that is where the headline sits.

TWO THINGS THIS CONSTRAINS

A banner carrying a picture is a dark band in both themes. `BannerSlider.vue` redefines the colour tokens under `.hero-dark`, so everything inside picks the dark set up through the classes it already uses. A light-mode page was laying a white wash over these images until they came out grey; shipping a second set for the other theme was the alternative, and this is cheaper and reads as intentional.

The headline transition must not be `mode="out-in"`. It was, which means the outgoing headline is removed and only then does the next one begin — the hero had no words in it at all for a third of a second every six seconds. Both slides now share one grid cell and cross-fade.

8.4

When a renewed term starts

`Subscription::continuesFromEndDate()`, and it is the only thing that decides this:

```
return $end !== null  
    && $end->copy()->addMonths($this->monthsOnPlan())->isFuture();
```

It lives on the model because **two very different things read it**: `RecordPayment`, which writes the dates, and `renewalTiming()`, which is what the notice on four screens promises. Those disagreeing is how a customer gets told one thing and charged for another — and they did disagree for a while, the notice still saying "restarts it from today" after the payment had started carrying on from the end date. `tests/Feature/Billing/RenewalDatesTest.php` renews at six different lapses and asserts the promised dates are the written ones.

A plan is a chain of periods and the chain should not have holes in it. A term runs from the day the last one ended, not from the day somebody got round to paying — which is also what the customer received, because the round keeps going through the grace period before anything is put on hold.

The condition is deliberately phrased as "*would this period still be running?*" rather than as a rule about status or a grace-period count. Those are settings that move; this cannot produce a plan that is expired on the day it is bought whatever anybody sets.

`test_a_renewed_period_is_never_already_over` holds that property across six lapse lengths.

WHAT IT USED TO DO

`period_end->isFuture() ? period_end : today()` — so any lapsed plan restarted at the payment date. A customer whose plan ran to the 10th and who paid on the 17th was charged a full month for 23 days of it, and their renewal date walked a week further into the month every year.

Taking money, and the sheet that covers it

Seven screens take payments. They all go through `shared/api/checkout.ts`, and that is what makes the progress sheet possible: `openThenPay` and `completeCheckout` move `paymentProgress` through four phases, and `shared/PaymentProgress.vue` — mounted once per bundle — renders the two that matter.

PHASE	COVERED?	WHY
opening	Yes	Our server is creating the payment. Nothing should be clicked twice.
gateway	No	Razorpay draws its own dialog. Covering it would block the thing the customer was asked to use.
confirming	Yes	The money is taken and we are verifying the signature. A refresh here leaves somebody charged and told nothing.
idle	No	Nothing in flight.

`confirming` also registers a `beforeunload` handler, so a reload during the one dangerous window asks first.

THE RULE FOR NEW PAYMENT SCREENS

Call a `payFor*` helper, or `openThenPay` for a new endpoint. A screen that posts and then calls `completeCheckout` itself will work, and will take somebody's money behind an unmarked page. And show the receipt when it comes back. Every helper returns `result.payment`; a confirmation that clears itself after a few seconds, or a panel that simply closes, reads to a customer exactly like being charged and told nothing.

Switching a whole feature off

Three of them today — `cloth_service`, `blog`, `team` — all named in `App\Support\Settings\Features`, which maps each to the setting behind it. A feature has to disappear from every door at the same moment, so adding one means all five of these:

WHERE	WHAT TO ADD
The setting	<code>SiteSettings::definitions()</code> , group Service, <code>'boolean' => true</code> . Then the constant and its mapping in <code>Features</code> .
The endpoints	<code>Route::middleware('feature:blog')</code> around the group. A 404, not a 403 — "switched off" and "does not exist" look the same from outside.
The masters	<code>'feature' => Features::BLOG</code> on the definition. <code>MasterRegistry::available()</code> then drops it from the menu and from <code>exists()</code> , which is what every master route checks.
The public site	<code>meta: { feature: 'blog' }</code> on the route, and the link in <code>PublicLayout</code> . The router guard awaits <code>whenFeaturesKnown()</code> .

The office

`meta: { feature: 'blog' }` on the route and `feature: 'blog'` on the navigation item; `auth.featureOn()` reads what `/me` sent.

TWO THINGS WORTH KEEPING STRAIGHT

An ability is not a feature. One says what this person may do, the other says what the business does at all. An administrator holds every ability there is, so a menu deciding by ability alone still offers them a blog that is switched off — and no amount of permission should bring it back.

The front end fails open, the server fails closed. An unknown feature is off in

`Features::on()`, so a typo in a route never quietly opens something up. The two front ends assume everything is on until told otherwise, because the alternative is a menu that sheds items on first paint and puts them back under somebody's cursor. The server is the one enforcing it either way.

WHY A SWITCH CAN LOOK LIKE IT DID NOTHING

The flags arrive with the session. The office reads them from `/me`, which is fetched once when the application starts, so saving a setting used to leave the menu offering a screen the server had just begun refusing — and the only cure was a full reload, which nobody thinks to do after pressing Save. `SiteSettingsView` now reloads the session as part of saving.

Or the browser is running an old bundle. With `npm run dev` stopped, the page falls back to whatever is in `public/build`, and every front-end change since that build is simply not there. Nothing warns you: the app works, it is just the wrong version of it. If a front-end change appears to have had no effect at all, check whether the served page references `:5173` or `/build/assets` before looking anywhere else.

8.7

The receipt, and the link that needs no account

Every captured payment sends `MessagePurpose::PaymentReceipt` from `RecordPayment::sendReceipt()`, carrying a signed link to `/receipt/{payment}` — a printable Blade page, not the SPA.

Public on purpose. Most customers never make an account, which is the same reason the public renewal page exists, so a receipt reachable only behind a session is one that turns into a phone call. The signature is the protection: generated with the application key, refused if a character changes. It never expires, because the message it arrives in does not either.

TWO TRAPS, BOTH SPRUNG ALREADY

Do not use route model binding here. It resolves through the sector scope, and nobody is signed in — so the scope covers nothing and every receipt link answered 404, valid ones included. The id is taken as a string and resolved inside `SectorContext::withoutScope()`. This is the third time that scope has caught an unauthenticated page.

Blade only sees a directive at a token boundary. `Received@if (...)` compiles as literal text and leaves the `@endif` with nothing to close, which is a 500 on the finished page and not a build error. Put a space or a newline before it.

The wording uses `{paid_amount}`, not `{amount}`. The second is what the plan costs today; a receipt has to say what was taken, on the day it was taken, under the invoice number it was taken under — and `MessageTemplate::valuesFor()` deliberately leaves the receipt placeholders blank rather than defaulting them from the plan, so a template that asks for them without a payment renders empty instead of wrong.

8.8

What goes stale when you save something

`resources/js/shared/api/refresh.ts`

Reference lists are cached hard — five minutes for the cleaners, the price list and the geography, because they are read on nearly every screen and change a few times a month. The cost is a cache that is right about everybody else's edits and wrong about your own.

Every screen used to clear only its own keys, which is exactly the half that was never the problem: the list in front of you refetched, and the four other screens reading the same records did not. Add a cleaner on People, walk to Subscriptions, and the picker did not offer them for another five minutes — which from the office's side means the cleaner was not saved, so they save them again.

```
// Not this
await queryClient.invalidateQueries({ queryKey: ['users'] })

// This – the fan-out is named once, in one file
await refreshAfter(queryClient, 'people')
```

Prefix matching does the rest: clearing `['cleaners']` also clears `['cleaners', <plan id>]`, so the per-plan pickers need no naming. The kinds are `people`, `masters`, `customers`, `subscriptions`, `payments`, `roles` and `cloth`; several can be passed at once, as the plan form does with `'subscriptions', 'customers'`.

A KEY THAT DOES NOT EXIST FAILS SILENTLY

`invalidateQueries` on a key nothing uses is not an error — it does nothing, quietly, and the screen it was supposed to refresh stays stale. The first draft of this file named `['payment']`; the detail panel actually uses `['payment-detail']`, so the open receipt would have gone on showing the old figures.

Worth checking a new entry against the real keys rather than trusting it:

```
grep -rhoE "queryKey: (computed\\(\\(\\) => )?\\['[a-z-]+' resources/js \\  
| grep -oE '\\['[a-z-]+' | tr -d '[' | sort -u
```

The front end

9.1

Two bundles

`vite.config.ts` — builds `site/main.ts` and `admin/main.ts` separately

A visitor reading the price list does not download the reports screen or the masters form. The **portal lives inside the admin bundle** because it needs the session and the auth store; it has its own layout and its own routes, and the router keeps the two apart by role, not by ability:

```
if (auth.isCustomer !== (to.meta.customer === true)) return home;
```

Three URL spaces, one blade decision

`routes/web.php`

```
Route::view('/login',      'app')->name('login.page');
Route::view('/app/{any?}', 'app')->where('any', '.*')->name('app');
Route::view('/my/{any?}',  'app')->where('any', '.*')->name('portal');

Route::view('/{any?}', 'site')
    ->where('any', '^(?!api|sanctum|up|storage|login|app|my).*')
    ->name('site');
```

THE TRAP

Any URL not in that exclusion list falls through to the **public** bundle, whose catch-all route redirects to the home page. When `/my` was missing here, every reload of a customer's page bounced them to the public home page and it looked like a session bug. If you add a URL space, add it to both the `Route::view` list *and* the negative lookahead.

9.2

Where logic goes

Not in `.vue` files. Anything with a decision in it — the signup flow, row selection, the subscriptions API — is a `.ts` module beside the views that use it (`site/signup.ts`, `admin/shared/useRowSelection.ts`, `admin/shared/subscriptions.api.ts`, `portal/portal.api.ts`, `shared/api/checkout.ts`). A component binds and renders; it does not decide.

This is applied to new work and to components as they are touched, not retrospectively across all of them. A bulk extraction was attempted and reverted; it broke 190 type checks in one go. Do it one component at a time, with the type check green between each.

Data fetching

TanStack Query throughout, with the query key naming the thing and its filters. Mutations invalidate rather than patching caches by hand. Axios lives in `shared/api/client.ts` and carries the Sanctum cookie.

Theme

One set of CSS custom properties in `resources/css/app.css`, light and dark, shared by both bundles. Use the token classes (`bg-surface`, `text-ink`, `border-line`, `text-accent`) rather than raw Tailwind colours, or the component will be wrong in one of the two themes.

Type checking

```
npx vue-tsc --noEmit
```

Run this rather than `npm run build` while working — `npm run dev` is normally already running and a build fights it.

Conventions

10.1

RULE	DETAIL
UUIDv7 keys	<code>BaseModel</code> brings <code>HasUuids</code> . Time-ordered, so they still index well, and an id in a URL reveals nothing about how many customers there are.
Integer paise	Columns end <code>_paise</code> . No floats, no decimals, formatted only at the edge.
Soft deletes	On <code>BaseModel</code> . Deleting hides; history stays joinable.
Audit columns	<code>HasAuditColumns</code> fills created-by and updated-by from the session.
Derived, not stored	"Expired" is a date comparison, not a column somebody has to remember to update. Same for the order type and the plan's standing.
Append-only ledgers	The cloth ledger is never edited. A correction is a new entry, and the weekly check proves the cached balance still matches the sum.
Sanitise on write	<code>Support/Content/HtmlSanitizer</code> , at the point of storing. Sanitising on read leaves the mess in the table for every future reader to remember to clean, and one that forgets is a hole.
Whitelist, do not blacklist	Settings keys, sortable columns and master types are all allow-lists. Anything not on the list is dropped, not stored.
Enums for state	Backed enums in <code>app/Enums</code> , cast on the model. No magic strings in controllers.
Comments say why	The codebase is heavily commented and the comments explain reasoning, not mechanics. When you change behaviour a comment describes, change the comment in the same edit.

Tests

11.1

```
php artisan test                # all 429
php artisan test --filter=AddSecondCarTest # one file
npx vue-tsc --noEmit            # the front end
```

Why the suite is fast

`tests/Concerns/MigratesOnlyWhenStale.php`

It hooks `setUpTraits()` and, when every migration on disk is already recorded in the test database, sets `RefreshDatabaseState::$migrated = true` so `migrate:fresh` is skipped. That took the suite from 217 seconds to roughly forty-seven. Add a migration and the next run rebuilds automatically — there is nothing to remember.

Both a schema dump and paratest were tried before this and made the suite slower — nine minutes. They are not the answer; do not reintroduce them.

The two tests that guard the rest

TEST	CATCHES
<code>Support/ClassNamesResolveTest</code>	Walks <code>app/</code> and fails if any referenced class name will not resolve. It exists because a missing <code>use Log;</code> made complaint creation return 500 <i>after</i> successfully creating the complaint.
<code>Api/MasterListingTest</code>	Walks <code>MasterRegistry</code> and opens every master, with and without filters. A master added later is covered the day it is added, without anybody remembering to come back.
<code>BranchScopeTest</code>	That the tenancy scope is still fail-closed.

Writing one

- `SectorContext::reset()` in both `setUp` and `tearDown`. Leaked branch state makes an unrelated test fail later and somewhere else.
- `fromSpa()` when the route needs the Origin header.
- Assert reads through `SectorContext::withoutScope()`, or the scope will hide the row you just created.
- Any paying route gets a "price is not taken from the request" test.

What has no automated cover yet

The cleaner app, the daily round and the reminder jobs. Worth knowing before you change any of them.

Commands and the schedule

12.1

COMMAND	DOES	SCHEDULED
<code>eswachh:backup</code>	Dumps the database, verifies it, prunes old copies. A failure raises a critical alert, not just a log line.	00:10, keep 14
<code>eswachh:reconcile-payments</code>	Settles attempts that never came back from the gateway.	00:20, <code>--days=7</code>
<code>eswachh:prune-service-history</code>	Deletes cleaning records past the retention period.	00:40
<code>eswachh:send-renewal-reminders</code>	Messages customers due or overdue.	09:30
<code>eswachh:hold-overdue</code>	Puts long-overdue plans on hold.	10:00, <code>--grace=7</code>
<code>eswachh:check-cloth-balances</code>	Verifies cached balances against the ledger. Exits non-zero on a mismatch.	Mon 06:00
<code>eswachh:check-integrations</code>	Reports whether payments and messaging are configured, and what is stopping them. <code>--ping</code> proves the key.	By hand
<code>eswachh:import</code>	Imports from the v1 database.	By hand
<code>eswachh:backfill-invoice-numbers</code>	Issues invoice numbers for captured payments that have none.	Called by the import

THE ORDERING IS A DESIGN DECISION

Backup first, so a later job going wrong leaves a copy from before it did. Settlement before reminders, so nobody is chased for money they already paid. Holds last, so they act on a corrected picture. Pruning after the backup, so a retention rule can never be the reason something is unrecoverable.

Every job is `withoutOverlapping`, `onOneServer` and `runInBackground`. Keep all three on anything you add: an unattended job that runs twice is worse than one that does not run at all.

Bugs this codebase has already had

- 13.1 Each of these cost real time to find. They are recorded because every one of them is easy to reintroduce.

The timezone

v1 ran on `Asia/Kolkata`; v2 started on UTC. Every imported timestamp was mislabelled by five and a half hours, so date filters found nothing and nobody could see why.

`APP_TIMEZONE=Asia/Kolkata` is not a preference.

Sign-in codes that never expired

`login_codes.expires_at` was a MySQL `TIMESTAMP`, which carries an implicit `ON UPDATE CURRENT_TIMESTAMP`. Every wrong guess touched the row and pushed the expiry forward, so a code lived forever. Migration `0001_01_01_000022` changes it to `DATETIME` and spends the outstanding ones. **Never use `TIMESTAMP` for an expiry.**

Two hundred customers in the wrong books

A sector was pointed at another franchise and nothing anybody could see changed. Every record carried its own `branch_id` — a copy of where the customer lived — and that copy was what the scope read, so the picker appeared to work while two hundred customers stayed put and the new owner signed in to a screen of zeroes.

A cache of a derived fact will drift, and nothing was comparing the two. The fix was to stop storing it: territory is now worked out from `customers.sector_id` against `user_sector` on every query. See section 03.

POWERSHELL WILL CORRUPT YOUR FILES

Two ways, both silent. `Set-Content -Encoding utf8` on 5.1 writes a **BOM**, and PHP will not parse a file that starts with one — 62 files at once. And a `-replace` round-trip through `ReadAllText` / `WriteAllText` mangled every non-ASCII character in eight more, turning ... into mojibake that a second "repair" pass made worse.

Use `sed`, which is byte-oriented, or the editor. If a file is already mangled, restore it from git rather than trying to decode it back.

Every master list at once

A variable that the listing closure never captured — `use ($definition, $filters)` where the body also wanted `$request`. One line, and it took out all twenty masters simultaneously: the price list, the geography, the banners, the templates. The screen said "Something went wrong" and the reason was only in the log.

PHP closures capture explicitly, so this class of mistake is silent until the line runs. It shipped because the Masters screen had *no test at all* — the single most load-bearing screen in the system, and nothing opened it. `MasterListingTest` now walks the registry.

₹225 on the page, ₹1,872 at the gateway

The public renewal page loaded the vehicle and the customer *inside* the sector scope. Nobody is signed in on that page, so the scope covers nothing and both relations came back `null` — and nothing threw. The price book, asked to price a plan with no car and no address, dropped the vehicle category and the society surcharge and answered with what was left.

A customer saw ₹225, pressed Pay, and Razorpay asked them for ₹1,872. The card above it showed an empty registration beside an empty name, which is the same bug and was the visible half nobody connected to the price.

THE GENERAL RULE

Anything reached without a session must resolve its records inside

`SectorContext::withoutScope()` — and that includes relations loaded *for* something else, like the two the price book reads. Fail-closed scoping is correct and did its job; the mistake was reading through it and then treating a null relation as a plan with no car.

A second defect sat behind it: the payment was opened from the plan's stored `amount_paise`, which is what it cost the last time it was bought, so quote and charge could disagree even once the quote was right. Renewals now price and write before `StartPayment` is asked.

`tests/Feature/Billing/PublicRenewalTest.php` pins both.

An editor that would not open at all

A `computed` declared above the query it reads. `societyId` referenced `existing` forty lines before `existing` was declared — which JavaScript allows right up until something evaluates it, and `useQuery` unrefs its `queryKey` during setup to build the first observer.

`ReferenceError: Cannot access 'existing' before initialization`, thrown while mounting, so the dialog never appeared and the Edit button read as dead.

The lesson is narrower than "declare things in order": **a reference inside an arrow function is deferred, but TanStack evaluates query keys eagerly**, so a key that reaches a later `const` is a runtime error rather than a lint warning. `vue-tsc` passes either way.

A form that opened blank the second time

The plan editor filled itself with `watch(existing, ...)` and no `immediate`. The dialog is behind a `v-if`, so it unmounts on close — and TanStack Query hands cached data over *during the first render* of the remounted component. The watcher is registered after that, nothing changes, and it never fires.

So: open a plan, close it, open the same plan again, and every dropdown reads Choose..., the car number is a dash, and the price panel says it needs a duration. Saving from there is the obvious next thing somebody does. It was reported as "cancel the payment window, then press Edit" because that sequence opens the dialog twice; it has nothing to do with payments.

WHERE THIS CAN BITE AGAIN

Any component that is unmounted and remounted and fills a form from a query keyed on an id. Panels that stay mounted are fine — the key changes and the data genuinely transitions. Adding `{ immediate: true }` costs one no-op call and removes the class, so put it on every one of them.

₹699 on the edit screen, ₹779 at the gateway

The plan editor's price panel quoted without `society_id`, so it priced the plan as though the customer lived nowhere in particular and dropped an ₹80 surcharge. Nothing on that screen mentions the society, so the office had two figures and no way to tell which was wrong.

This is the third time the same omission has shipped — the public renewal page and the customer's renew panel both had it. **Any quote for an existing customer needs the society.**

`PriceBook::forRenewal()` reads it from the customer and is the safe path; a hand-built call to `/pricing/quote` is where it goes missing.

The missing use statement

A complaint was created and then the notification's own catch block threw, because `Log` was not imported. The customer saw a 500 and the complaint existed. `ClassNamesResolveTest` now walks `app/` and fails on any name that will not resolve.

Every portal reload going home

`/my/*` was not listed in `routes/web.php`, so it fell through to the public bundle whose catch-all redirects home. Covered in section 09; it was reported twice as a session bug before the cause was found.

A cleared setting reverting to its default

`SiteSettings::read()` used `??`, which cannot tell a stored `null` from a missing row. Emptying a policy page and saving brought the shipped wording straight back, and looked like the save had failed. It uses `array_key_exists` now.

A settings cache that hid new settings

The SEO fields were added and the site kept serving the fallback title until somebody ran `cache:clear` by hand. The cache now rebuilds itself when the number of definitions changes.

Imported payments with no invoice number

All 492 of them. `eswachh:backfill-invoice-numbers` exists for this and the importer calls it.

A dropdown clipped by its own container

The plan list's action menu was cut off by an `overflow-x-auto` ancestor. It is teleported to `<body>` with fixed positioning and repositioned on scroll and resize. If you add a menu inside a scrolling table, do the same.

And one about tooling, not the app

DO NOT EDIT PHP THROUGH SHELL HEREDOCs OR NODE SCRIPTS

Passing PHP through a shell strips `$` sigils and `\` namespace separators, silently producing `AppModelsPayment`, `->actingAs(->owner)` and similar. This has caused real bugs in this repository, including the missing-import one above. Edit files with an editor.

House rules for this repository

- **Do not commit.** The owner commits manually.
- **Do not run** `npm run build` during development — `npm run dev` is normally already running. Use `npx vue-tsc --noEmit`.
- **Finish the whole task, then run** `php artisan test` **once**. Not after each step.

Still open

- Cloth count selectable while renewing on the *public* renew page.
- Pending renewals and cloths-below-five as dedicated pages rather than saved filters.
- Society-wise WhatsApp targeting could be more direct than it is.
- An "are you sure?" confirmation before a cleaner marks a car cleaned.
- No automated tests for the cleaner app, the daily round or the reminders.
- Five v1 orders (₹3,045) cannot be imported; their customers were deleted in v1.
- Extracting logic out of the remaining `.vue` components — parked by the owner, one of twenty-one done.

Cutting over from v1

- 14.1 v1 is still live and still taking orders, and features have been added to it since v2 was started. Two different questions follow from that, and only one of them is real work.

The code does not need porting

v1's newest migrations are complaints, cleaner attendance, `sector_user`, the franchise-owner role, and payment tracking on `payment_history`. Every one of those already exists in v2, built natively rather than carried over — sector scoping is the whole tenancy model here, complaints and attendance have their own screens, and payments have carried gateway ids since the first day.

So there is no v1 code to migrate. What is left is a data question: does the content of those newer tables come across? Today it does not — see 14.3.

Catching up the data is one command, run again

The importer is idempotent, and this was checked rather than assumed. Importing the same database twice:

```
after 1st import: customers=212 subs=222 payments=492 map=1519
after 2nd import: customers=212 subs=222 payments=492 map=1519
```

And importing an older snapshot, then a newer one on top of it — which is exactly what a second run against a still-live v1 looks like:

```
older snapshot:    customers=203 subs=213 payments=481
newer on top:      customers=212 subs=222 payments=492
duplicate invoices: 0
```

The delta arrives and nothing doubles. `legacy_references` is what makes that true: every record the importer creates is written there against its v1 id, so a second run finds the mapping and updates rather than inserts.

14.2

The sequence on the day

1. **Take a fresh dump of the production v1.** Not one of the local snapshots — they are months apart from each other, let alone from production, and picking the wrong one silently imports an old world.
2. **Load it locally and point `LEGACY_DB_DATABASE` at it.** The command refuses anything without an `orders` table, so a v2 database by mistake stops here rather than half way through.
3. `php artisan eswachh:import --dry-run` and read the skipped table. Every payment reporting "no matching subscription" is normal in a dry run — nothing was written, so nothing is in the map yet.
4. `php artisan eswachh:import`. Existing records update; new ones are added.

5. **Check the counts** against v1's own: `orders` against `subscriptions`, `payment_history` against `payments`.
6. **Stop v1 taking orders before the final run**, or the last few minutes of trading are stranded in a system nobody is watching any more.

REHEARSE IT, THEN REHEARSE IT AGAIN

Because it is idempotent, the run on cutover day should be the fiftieth time you have done it, not the first. Import a fresh dump weekly in the run-up: it costs a command, and it turns the day itself into something boring.

14.3

What the importer does not bring

Eight v1 tables hold data and are not read. Whether that matters is a business decision, not a technical one, so it is listed rather than assumed:

V1 TABLE	ROWS	WHAT IT IS, AND WHETHER IT IS WORTH CARRYING
<code>order_history</code>	465	Service history. v2 prunes its own after fifty days, so importing years of it fills a table that empties itself.
<code>posts</code>	25	Blog articles. Currently switched off for launch — worth importing only when the blog goes on.
<code>comments</code>	25	Blog comments, and they belong with the articles.
<code>tags / taggables</code>	10 / 189	Blog tags, likewise.
<code>teams</code>	9	The team page. Nine records — quicker to retype than to write an importer for.
<code>settings</code>	23	v1's business settings. v2's are different and richer; enter them by hand and check each one.
<code>complaints</code>	1	Added to v1 recently. One row.
<code>cleaner_attendances</code>	0	Added to v1 recently. Empty.

Adding a step for any of these is an afternoon: a method on `ImportLegacyData`, an entry in `--step`, and `upsert()` to keep it idempotent like the rest.

TWO DATA PROBLEMS IN V1 WORTH FIXING IN V1

38 customer profiles point at a society that no longer exists — ids above 7, left behind when v1 moved from `societys` to `societies`. Twenty of them have orders. Those customers import with no society, which means no surcharge in their price. Fixing it in v1 before the cutover is far easier than reconciling it afterwards.

Seven car numbers are registered to more than one customer. The public renewal page finds a plan by car number and takes the first match, so one of those customers reaches the other's plan. This is v1 data, not an import artefact — it appears after a single clean run.